CMPSC 463

Algorithms and Data Structures

Algorithmic Time-Series Segmentation and Condition Analysis Using Water Pump RUL Dataset

Submitted By: Xiomara Mohamed

GitHub: https://github.com/xio0x/Project-1

Submitted On: 15 March 2026

Table of Contents

1. Project Overview 3
2. Installation and Usage Instructions 4
3. Code Structure Explanation 5
4. Algorithm Descriptions 6
5. Toy Example Verification 9
6. Dataset Description 11
7. Execution Results 12
8. Discussion and Conclusions 18

1. Project Overview

The project presents an algorithm-driven system for segmenting, clustering, and analyzing real-world time-series sensor data collected from a water pump machine. The dataset comprises 10,000 time instances, 52 sensor channels, and a Remaining Useful Life (RUL) measurement recorded in hours.

Three core algorithmic techniques are applied: Divide-and-Conquer segmentation, Divide-and-Conquer clustering, and Kadane's Maximum Subarray algorithm. These methods characterize machine behavior and relate findings to four machine-health categories derived from the RUL field.

No machine learning libraries are utilized; all algorithms are implemented from scratch using Python, NumPy, Pandas, and Matplotlib.

Machine-Health Categories (Derived from RUL)

The continuous RUL field is split into four categories computed from my dataset: Q10 (10th percentile), Q50 (50th percentile and/or median), and Q90 (90th percentile).

Category	Definition
Extremely Low RUL	rul < Q10 (10th percentile)
Moderately Low RUL	Q10 <= rul < Q50 (50th percentile)
Moderately High RUL	Q50 <= rul < Q90 (90th percentile)
Extremely High RUL	rul >= Q90

2. Installation and Usage Instructions

Requirements

- Python 3.7 or higher
- pip (Python package manager)
- Dataset file: rul_hrs.csv (download from Kaggle — link in Section 6)

Step 1: Clone the Repository

```
git clone https://github.com/xio0x/Project-1.git
cd Project-1
```

Step 2: Install Dependencies

```
pip install -r requirements.txt
```

The requirements.txt file includes: pandas, numpy, matplotlib

Step 3: Download the Dataset

Download the Water Pump RUL – Predictive Maintenance dataset from Kaggle and place rul_hrs.csv in the same folder as main.py:

https://www.kaggle.com/datasets/anseldsouza/water-pump-rul-predictive-maintenance

Step 4: Run the Project

```
python3 main.py
```

Output files are saved to the outputs/ directory:

- outputs/task1/ - 10 segmentation plots (.png) and task1_summary.txt
- outputs/task2/ - task2_summary.txt
- outputs/task3/ - task3_summary.txt

3. Code Structure Explanation

The project is organized into five Python modules, each with a single responsibility:

File	Responsibility
main.py	Entry point. Loads data, assigns RUL categories, runs toy example verification, then calls each task in sequence.
utils.py	Data loading, quantile-based RUL category assignment, and toy example verification that actually executes each algorithm on small known inputs.

segmentation.py	Recursive Divide-and-Conquer segmentation algorithm and helpers for Task 1, including plot saving and per-segment RUL summarization.
clustering.py	Recursive top-down divisive clustering from scratch for Task 2. Tracks original row indices to ensure correct label retrieval.
kadane_analysis.py	Kadane's Maximum Subarray algorithm for Task 3. Preprocesses sensor signals and runs the algorithm across all 52 channels.

Data Flow

1. main.py loads rul_hrs.csv and slices the first 10,000 rows.
2. assign_rul_categories() computes Q10, Q50, Q90 and appends a rul_category column.
3. print_toy_example_checks() runs each algorithm on tiny arrays to verify correctness.
4. run_segmentation_task(df) processes 10 sensors and saves segmentation plots.
5. run_clustering_task(df) clusters all 10,000 rows into 4 groups.
6. run_kadane_task(df) applies Kadane's algorithm to all 52 sensor channels.

4. Algorithm Descriptions

Task 1: Divide-and-Conquer Segmentation

The segmentation algorithm recursively partitions a time-series signal into stable and unstable regions based on variance.

Algorithm steps: Compute the variance of the current segment signal (from start to end). List all steps that follow, including threshold checks, recursion, and final segment marking.

1. If variance exceeds the threshold and the segment is longer than the minimum length (16 elements), split at the midpoint and recurse on both halves.
2. Otherwise, mark the segment as stable and record it as a final segment.

The threshold is set to 35% of the signal's global variance, making it scale-adaptive per sensor. The minimum segment length of 16 prevents excessive recursion depth. After segmentation, a Segmentation Complexity Score (total number of segments) is computed, and each segment is labeled with its majority RUL category.

Task 2: Divide-and-Conquer Clustering

A recursive top-down (divisive) clustering algorithm groups the 10,000 time instances into 4 clusters based on their 52-dimensional sensor readings without using any machine learning library.

Algorithm steps: Begin with one cluster containing all row indices (0 to 9,999). Then, sequentially identify the largest cluster, compute its centroid, select the farthest point as the second seed, assign points to clusters by squared Euclidean distance, replace split clusters, and repeat until four clusters are formed.

1. Identify the largest current cluster and split it.
2. Compute the centroid of the selected cluster.
3. Find the point farthest from the centroid; this becomes the second seed.
4. Assign every point to whichever seed it is closer to using squared Euclidean distance.
5. Replace the split cluster with the two resulting sub-clusters.
6. Repeat until 4 clusters are formed.

Row indices are tracked throughout the process rather than copying data arrays, avoiding incorrect label assignments when duplicate sensor readings exist.

Task 3: Maximum Subarray (Kadane's Algorithm)

Kadane's algorithm detects the contiguous subarray of maximum sum within a preprocessed sensor signal, finding the time interval with the most intense sustained deviation from baseline behavior.

Preprocessing per sensor:

1. Compute the absolute first-difference: d[i] = | sensor[i] - sensor[i-1] |
2. Mean-center the differences: x[i] = d[i] - mean(d)

Kadane's algorithm: Initialize max_sum and current_sum to x[0]. For each subsequent element, compare extending the current subarray or starting afresh, and update the best indices if a new maximum sum is found. Final steps include examining category results.

1. For each subsequent element, either extend the current subarray or start fresh at the current index, whichever gives the higher running sum.
2. Track the best start and end indices whenever a new maximum is found.

After finding the max-deviation interval, the RUL categories of the time indices within it are examined. Sensors whose high-deviation windows are dominated by low-RUL categories are candidate early-warning indicators of machine failure.

5. Toy Example Verification

Before processing the real dataset, each algorithm was verified on small hand-crafted inputs with known expected outputs. These checks run automatically every time main.py is executed.

5.1 Segmentation Toy Example

Input signal (32 elements):

[1,1,1,1,1,1,1,1,1,1,1,1,1,1,1,1, 10,1,10,1,10,1,10,1,10,1,10,1,10,1,10,1]

The first 16 elements are near-constant (low variance). The second 16 alternate between 1 and 10 (high variance). The algorithm correctly produces 2 segments:

- Segment 1: [0, 16) : stable low-variance region kept whole
- Segment 2: [16, 32) : high-variance region identified as unstable

Complexity score: 2. This confirms the algorithm correctly distinguishes stable from volatile regions.

5.2 Clustering Toy Example

Input: 4 points in 2D space: two near [1,1] and two near [9,9]:

[[1.0, 1.0], [1.5, 1.5], [9.0, 9.0], [9.5, 9.5]]

Requesting 2 clusters, the algorithm correctly separates them:

- Cluster 1: [[9.0, 9.0], [9.5, 9.5]]
- Cluster 2: [[1.0, 1.0], [1.5, 1.5]]

The farthest-point seeding correctly separated the two groups, confirming the split logic and index-tracking work.

5.3 Kadane Toy Example

Classic Kadane verification with a known answer:

Input: [-2.0, 3.0, 4.0, -1.0, 2.0, -5.0]

Expected: subarray [3.0, 4.0, -1.0, 2.0] at indices [1, 4], sum = 8.0

Actual result: best start = 1, best end = 4, values = [3.0, 4.0, -1.0, 2.0], sum = 8.0 : PASSED. The code's assertion confirmed the exact expected values.

6. Dataset Description

6.1 Source

Dataset: Water Pump RUL Predictive Maintenance (Kaggle)

URL: https://www.kaggle.com/datasets/anseldsouza/water-pump-rul-predictive-maintenance

This is a real-world industrial dataset collected from a water pump. The project uses the first 10,000 rows and all 53 columns.

6.2 Variables Used

Variable	Type	Description
sensor_00 – sensor_51	Float (52 cols)	Real-valued sensor measurements at each time step.
rul	Float	Remaining Useful Life in hours/time until expected failure.
rul_category	String (derived)	Four-class health label added at runtime using quantile boundaries.

6.3 RUL to 4-Category Transformation

Quantile boundaries computed from the first 10,000 rows of the dataset:

Quantile	Value	Boundary Role
Q10 - 10th percentile	135.9317 hours	Separates Extremely Low from

		Moderately Low RUL
Q50 - 50th percentile	202.5917 hours	Separates Moderately Low from Moderately High RUL
Q90 - 90th percentile	269.2517 hours	Separates Moderately High from Extremely High RUL

7. Execution Results

7.1 Task 1: Divide-and-Conquer Segmentation

Ten sensors (sensor_00 through sensor_09) were segmented using the recursive Divide-and-Conquer algorithm with a threshold of 35% of each signal's global variance and a minimum segment length of 16.

Sensor	Complexity Score	Dominant RUL Category in Segments
sensor_00	19	Moderately High RUL (early), Extremely Low RUL (late)
sensor_01	24	Moderately High RUL (early), Extremely Low RUL (late)
sensor_02	17	Moderately High RUL (early), Extremely Low RUL (late)
sensor_03	30	Moderately High RUL (early), Extremely Low RUL (late)
sensor_04	974	Extremely High / Moderately High RUL (early), Extremely Low RUL (late)

sensor_05	168	Moderately High RUL (early), Extremely Low RUL (late)
sensor_06	76	Extremely High / Moderately High RUL (early), Extremely Low RUL (late)
sensor_07	77	Moderately High RUL (early), Extremely Low RUL (late)
sensor_08	397	Extremely High / Moderately High RUL (early), Extremely Low RUL (late)
sensor_09	15	Moderately High RUL (early), Moderately Low RUL (late)

Segmentation plots for all 10 sensors are shown below. Red dashed lines indicate segment boundaries detected by the algorithm.

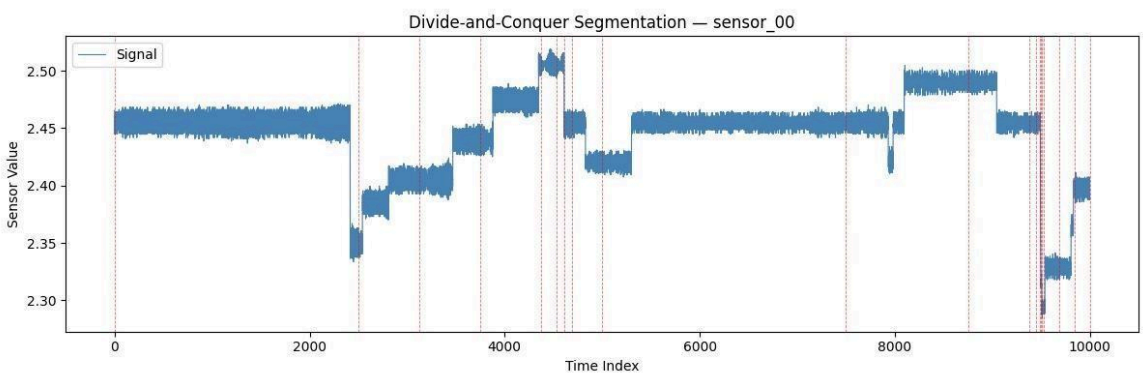

Figure 1: sensor_00 segmentation (Complexity Score: 19). Clear level shifts around index 2500 and 9000 coincide with RUL category transitions.

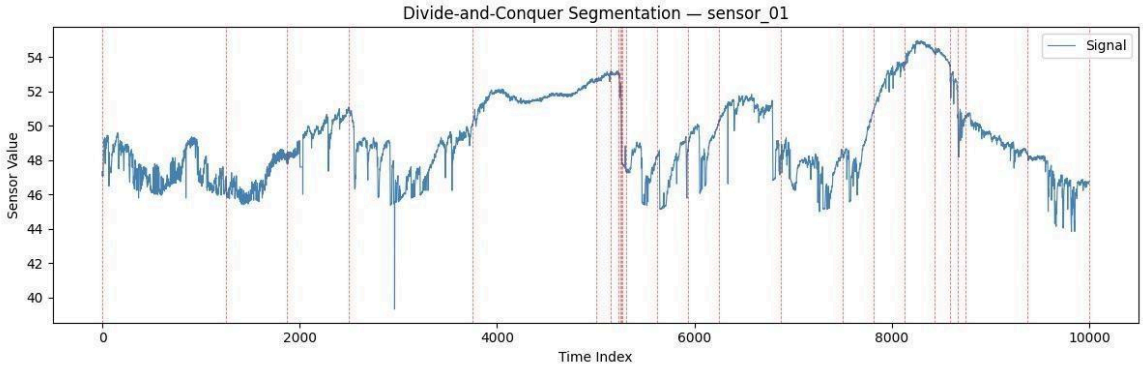

Figure 2: sensor_01 segmentation (Complexity Score: 24). A gradual trend with a sharp drop near index 4500 triggers dense edge detection.

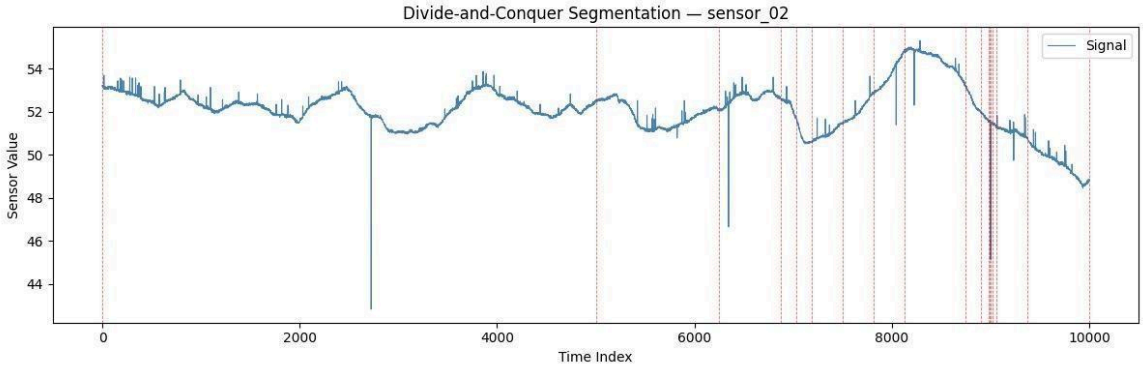

Figure 3: sensor_02 segmentation (Complexity Score: 17). A smooth decline in the second half with segment boundaries tracking the downward trend.

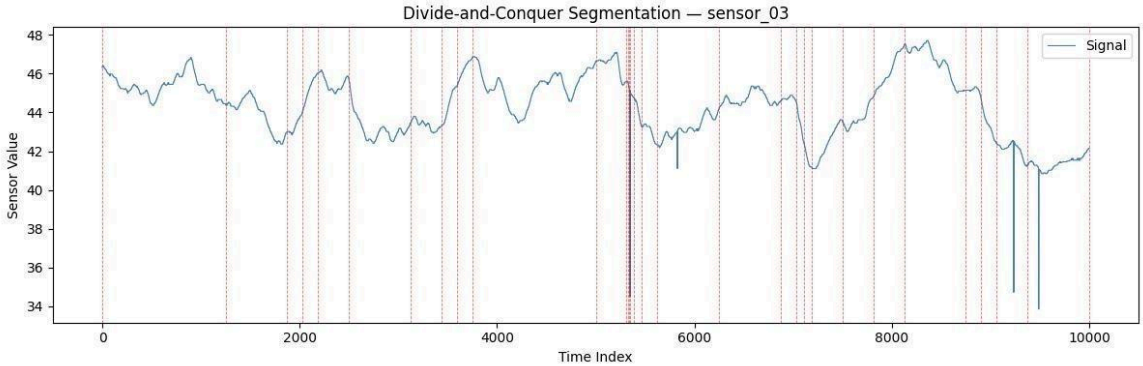

Figure 4: sensor_03 segmentation (Complexity Score: 30). An oscillating signal with clustered boundaries near index 5000, where variance peaks.

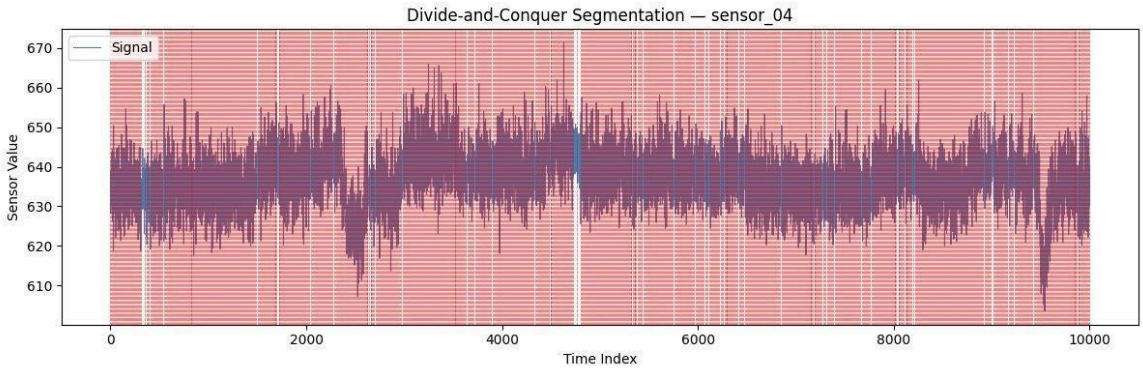

Figure 5: sensor_04 segmentation (Complexity Score: 974). The highest complexity score by far; a near-constant high-frequency signal causes maximum fragmentation throughout.

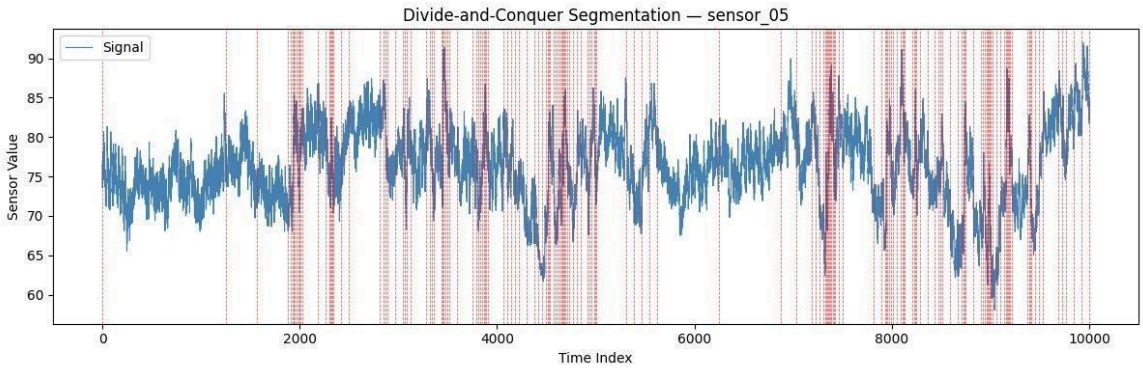

Figure 6: sensor_05 segmentation (Complexity Score: 168). The broad oscillations with denser boundary clusters in high-variance regions around index 2000 and 8000.

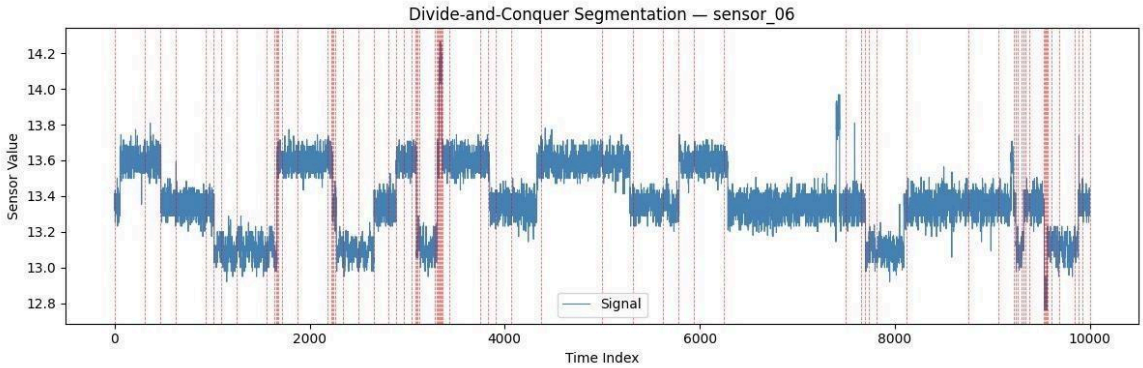

Figure 7: sensor_06 segmentation (Complexity Score: 76). A step-like behavior with boundaries grouping around the major level transitions.

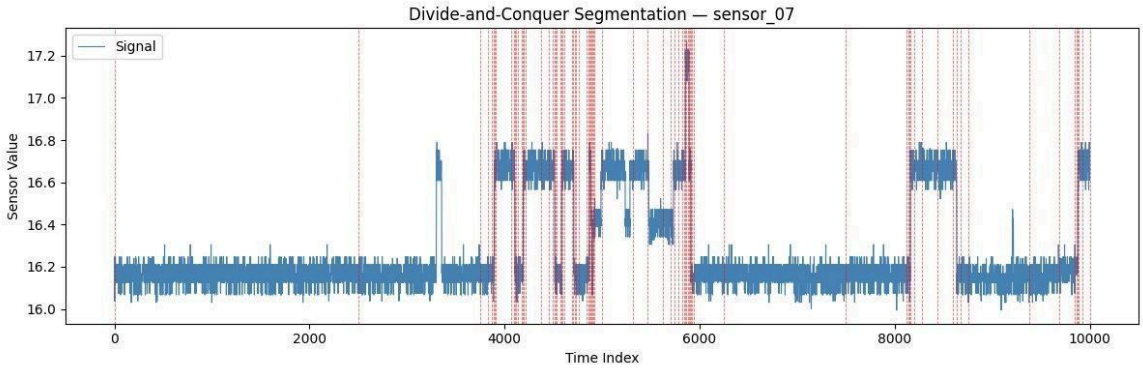

Figure 8: sensor_07 segmentation (Complexity Score: 77). Almost stable baseline with a concentrated burst of boundaries between index 4000 and 6000.

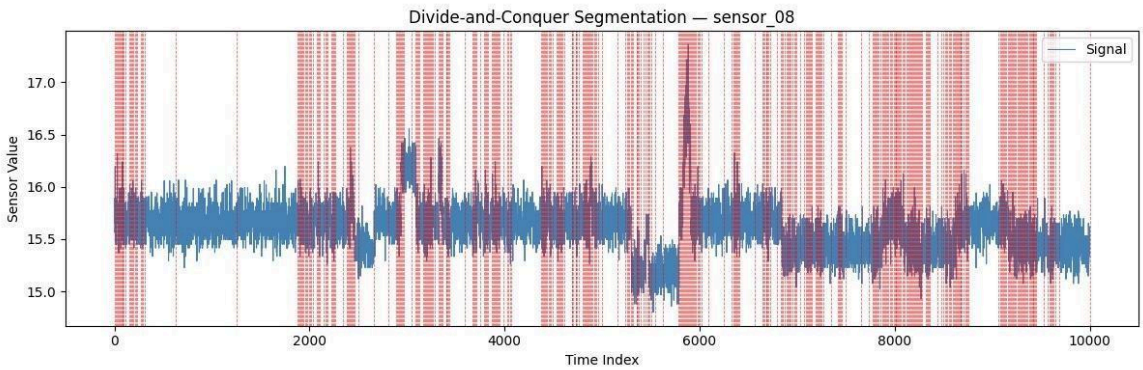

Figure 9: sensor_08 segmentation (Complexity Score: 397). A high-frequency noisy signal with dense boundaries throughout; second-highest complexity score.

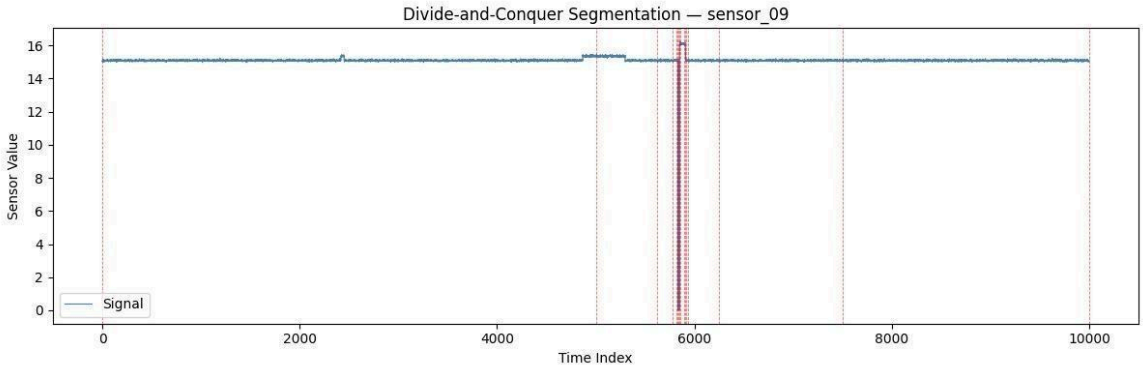

Figure 10: sensor_09 segmentation (Complexity Score: 15), the lowest complexity score; nearly flat signal with one brief spike near index 5800, causing local fragmentation.

7.2 Task 2 : Divide-and-Conquer Clustering

The divisive clustering algorithm was applied to all 10,000 time instances (52 sensor features each), producing 4 clusters.

Cluster	Size	Majority RUL Class	Class Counts
Cluster 1	116	Moderately Low RUL	Mod. High: 3, Mod. Low: 113
Cluster 2	82	Moderately Low RUL	Mod. Low: 82

Cluster 3	9,725	Moderately High RUL	Ext. High: 923, Mod. High: 3997, Mod. Low: 3805, Ext. Low: 1000
Cluster 4	77	Extremely High RUL	Ext. High: 77

7.3 Task 3 : Maximum Subarray (Kadane)

Kadane's algorithm was applied to the preprocessed difference signal of all 52 sensor channels. Results for all sensors are shown below.

Sensor	**Interval**	**Deviation**	**Dominant RUL Category**
sensor_00	[62, 4830]	9.49	Moderately High RUL
sensor_01	[1, 2038]	184.01	Moderately High RUL
sensor_02	[8990, 9825]	19.57	Extremely Low RUL
sensor_03	[5217, 9488]	32.34	Moderately Low RUL
sensor_04	[736, 5914]	1042.73	Moderately High RUL
sensor_05	[102, 4789]	859.84	Moderately High RUL
sensor_06	[2689, 4488]	7.66	Moderately High RUL
sensor_07	[3884, 9996]	11.37	Moderately Low RUL

sensor_08	[5767, 8749]	24.51	Moderately Low RUL
sensor_09	[5832, 5919]	61.47	Moderately Low RUL
sensor_10	[2786, 9994]	313.19	Moderately Low RUL
sensor_11	[1835, 7862]	97.07	Moderately High RUL
sensor_12	[1870, 2959]	108.93	Moderately High RUL
sensor_13	[5857, 9974]	1024.12	Moderately Low RUL
sensor_14	[8622, 9994]	354.07	Extremely Low RUL
sensor_16	[2018, 4014]	927.50	Moderately High RUL
sensor_17	[7630, 9998]	4372.13	Moderately Low RUL
sensor_18	[7904, 9994]	46.31	Moderately Low RUL
sensor_19	[8717, 9998]	687.99	Extremely Low RUL
sensor_20	[1797, 5398]	419.38	Moderately High RUL
sensor_21	[934, 6525]	1938.04	Moderately High RUL
sensor_22	[8599, 9997]	1209.99	Extremely Low RUL
sensor_23	[267, 3556]	1067.32	Moderately High RUL
sensor_24	[8600, 9998]	3682.63	Extremely Low RUL
sensor_25	[5332, 9997]	4152.69	Moderately Low RUL

sensor_26	[273, 2639]	878.95	Moderately High RUL
sensor_27	[334, 622]	10840.48	Extremely High RUL
sensor_28	[242, 993]	3619.98	Extremely High RUL
sensor_29	[5061, 9998]	4935.52	Moderately Low RUL
sensor_30	[882, 2442]	10405.23	Moderately High RUL
sensor_31	[830, 4075]	16526.53	Moderately High RUL
sensor_32	[4197, 8179]	5165.27	Moderately Low RUL
sensor_33	[241, 5542]	3681.80	Moderately High RUL
sensor_34	[2026, 6659]	1761.85	Moderately High RUL
sensor_35	[3450, 9997]	2681.12	Moderately Low RUL
sensor_36	[3545, 9998]	4095.08	Moderately Low RUL
sensor_37	[8808, 9998]	4352.52	Extremely Low RUL
sensor_38	[3110, 9842]	276.13	Moderately Low RUL
sensor_39	[8222, 9561]	154.85	Moderately Low RUL
sensor_40	[7332, 9112]	1776.44	Moderately Low RUL
sensor_41	[2868, 9937]	139.94	Moderately Low RUL
sensor_42	[7326, 9893]	100.36	Moderately Low RUL

sensor_43	[7337, 9299]	880.28	Moderately Low RUL
sensor_44	[172, 3129]	411.02	Moderately High RUL
sensor_45	[2170, 3140]	1190.37	Moderately High RUL
sensor_46	[7268, 9782]	1056.75	Moderately Low RUL
sensor_47	[4852, 9896]	325.67	Moderately Low RUL
sensor_48	[5586, 7602]	1321.92	Moderately Low RUL
sensor_49	[19, 6922]	1271.96	Moderately High RUL
sensor_51	[4577, 7868]	630.02	Moderately Low RUL

Sensors whose max-deviation interval is dominated by low-RUL categories (potential early indicators of machine degradation):

sensor_02, sensor_03, sensor_07, sensor_08, sensor_09, sensor_10, sensor_13, sensor_14, sensor_17, sensor_18, sensor_19, sensor_22, sensor_24, sensor_25, sensor_29, sensor_32, sensor_35, sensor_36, sensor_37, sensor_38, sensor_39, sensor_40, sensor_41, sensor_42, sensor_43, sensor_46, sensor_47, sensor_48, sensor_51

8. Discussion and Conclusions

8.1 Interpretation of Findings

Task 1 : Segmentation:

The segmentation results demonstrate a consistent pattern across most sensors. Early segments correspond to the Moderately High or Extremely High RUL range, indicating a healthy machine, while later segments transition to the Moderately Low or Extremely Low RUL range, reflecting machine degradation. This temporal alignment indicates that the Divide-and-Conquer algorithm reliably identifies structural changes in the signal that correspond to actual transitions in machine health.

Sensor_04 has a complexity score of 974, which is substantially higher than that of all other sensors. The corresponding plot shows near-constant high-rate noise throughout the window, leading the algorithm to fragment the signal into hundreds of minimum-length segments. This observation suggests that sensor_04 measures an inherently noisy variable, such as vibration or pressure fluctuations, rather than a gradually changing health indicator. In contrast, sensor_09 has the lowest score of 15, displaying a nearly flat signal with only one brief anomaly near index 5800. This sensor demonstrates minimal temporal variation and may be less informative for health monitoring.

Task 2 : Clustering:

The divisive clustering algorithm produced a highly imbalanced result. Cluster 3 contained 9,725 of the 10,000 rows, while the remaining three clusters included only 116, 82, and 77 rows, respectively. The largest cluster spans all four RUL categories, indicating that the majority of sensor data does not separate distinctly in the 52-dimensional space using this method. The three smaller clusters are notably purer: Cluster 4 consists entirely of Extremely High RUL (77 rows), and Clusters 1 and 2 are dominated by Moderately Low RUL. These findings suggest that the algorithm effectively identified a small set of extreme-condition readings at both ends of the health spectrum, while most mid-range readings remain too similar to be separated by divisive splitting alone.

Task 3 : Kadane:

Twenty-nine of the 52 sensors have their maximum deviation intervals dominated by low-RUL categories (Moderately Low or Extremely Low RUL). This finding indicates that more than half of the sensors exhibit their most intense activity during the later, degraded phase of the machine's operational life. Sensors with deviation windows concentrated at high time indices, such as sensor_02 at [8990, 9825], sensor_14 at [8622, 9994], and sensor_22 at [8599, 9997], are particularly valuable as late-stage indicators, as they spike when the machine approaches failure. In contrast, sensors whose deviation intervals fall in the Extremely High RUL zone (e.g., sensor_27, sensor_28) exhibit their highest activity early in the machine's life and are ineffective as failure predictors.

8.2 Challenges

- Recursive depth limits: The recursive segmentation algorithm approached Python's default recursion limit on sensor_04 and sensor_08 due to their persistent high variance. The minimum segment length of 16 prevented stack overflow but resulted in very high complexity scores for these sensors.
- Threshold selection: Using 35% of each sensor's global variance is adaptive but still arbitrary. Sensors with uniformly high variance (like sensor_04) get fragmented regardless of the threshold because no segment ever falls below it.

- Clustering imbalance: The divisive algorithm consistently produced one very large cluster and three tiny ones. This is a known weakness of top-down clustering — it cannot merge points once they are split, so early splits that separate outliers leave the bulk of the data in a single undivided group.
- Index alignment in Kadane: The np.diff() output is one element shorter than the starting signal. Careful index mapping was required to correctly retrieve RUL category labels for the identified deviation interval.

8.3 Limitations and Possible Improvements

- Threshold tuning: A data-driven threshold (e.g., based on the median absolute deviation or cross-validation) could yield more meaningful and consistent segmentation across sensors with different noise profiles.
- Clustering quality: The imbalanced cluster sizes suggest that divisive clustering is not well-suited to this dataset's geometry. K-means with iterative tuning or an agglomerative approach would likely yield more balanced, interpretable clusters.
- Sensor coverage in Task 1: Only 10 of 52 sensors were analyzed in the segmentation task. Including all 52 would provide a more complete picture of which channels show the most useful time dynamics.
- Variate Kadane: Running Kadane independently on each sensor misses coordinated multi-sensor deviation patterns. A composite signal (e.g., a weighted average across all 52 sensors) could identify globally significant deviation windows across the machine.
- Visualization: Adding RUL category color-coding to the segmentation plots (e.g., shading each segment by its dominant category) would make health-state transitions immediately visible without consulting the summary file.